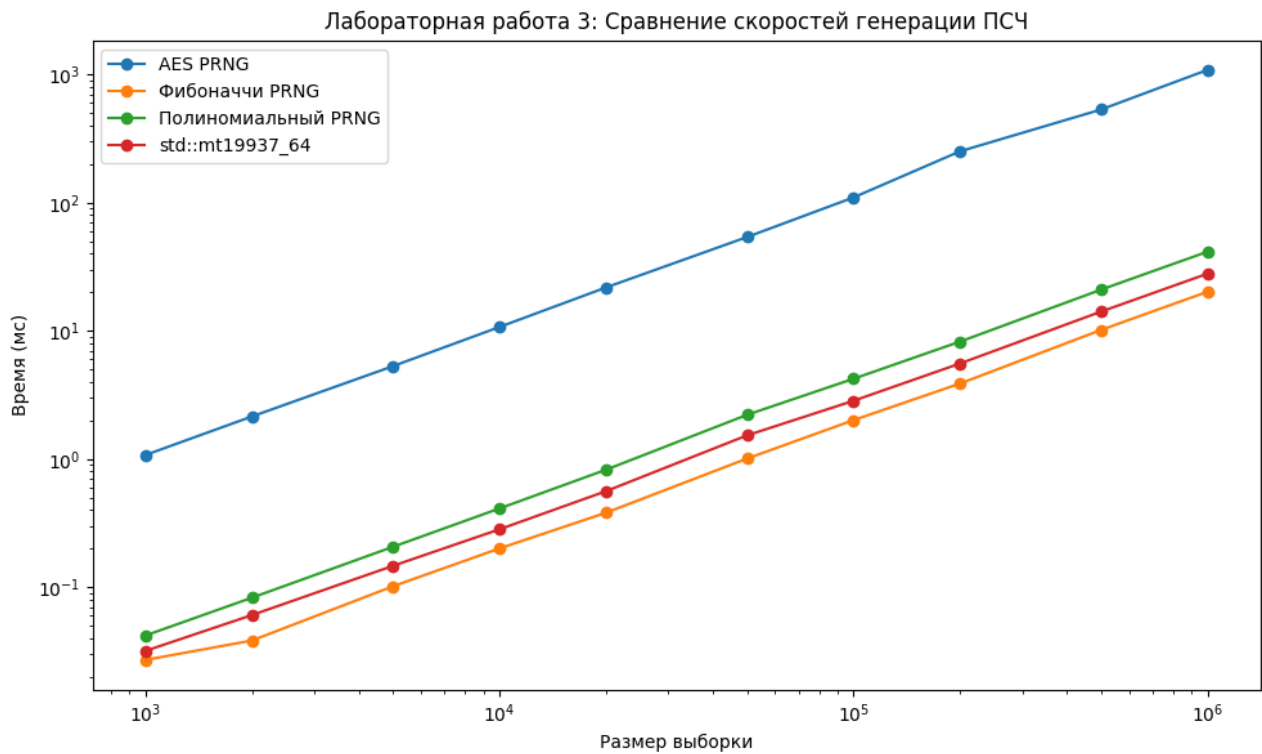


Лабораторная работа 3



Репозиторий: <https://github.com/diduk001/programming-techniques/tree/main/practice3>

Лабораторная работа 3: Генератор псевдослучайных чисел

Общее задание:

- 1) Модифицировать (предложить собственные) три метода генерации псевдослучайных чисел.
- 2) Получить не менее 20 выборок каждым методом (диапазон чисел в каждой выборке не менее 5000) объемом не менее 1000 элементов каждая.
- 3) Для каждой выборки посчитать среднее, отклонение и коэффициент вариации.
- 4) Каждую выборку проверить на равномерность распределения и случайность, используя критерий Хи-квадрат. Сделать выводы о равномерности и случайности генератора.
- 5) Для каждого алгоритма осуществить проверку с помощью не менее 5-ти тестов NIST и/или Diehard. Ссылки на реализацию тестов можно найти тут. Сделать выводы и сравнить их с п.4.
- 6) Засечь время генерации чисел от тысячи до миллиона элементов обоими предложенными методами и любым стандартным методом используемого языка программирования. Построить графики сравнения скоростей в зависимости от объема выборки.

Выбранные алгоритмы:

1) Предложенный метод генерации с использованием элементов AES:

1. Задан параметр Key - матрица байт 4x4 с заранее случайно сгенерированными числами, S - таблица замены, где для каждого байта определено значение, на которое он заменяется
2. Изначальное состояние (seed, 16 байт) записывается в матрицу 4x4. Назовем матрицу состояния State
3. 10 раз повторяется следующее:
 1. Операция SubBytes заменяет каждый байт из State на соответствующий байт согласно S
 2. ShiftRows циклически сдвигает влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3
 3. AddKey поэлементно прибавляет Key к State используя XOR
 4. Состояние возвращается как псевдослучайное число

2) Предложенный метод генерации на основе чисел Фибоначчи

1. Задан параметр A - inicialный параметр генератора
2. Изначальный seed - число, кладется в переменную B
3. Состояние вычисляется согласно модифицированной формулы чисел Фибоначчи: $A, B = B, (A * A) + ((B * B) \ll 17 \mid (B * B) \gg 47)$
4. B возвращается как псевдослучайное число

3) Предложенный метод генерации на основании полиномиального хэширования

1. Задается seed - 8 байт
2. seed побайтово складывается с заранее заданными случайными 8 байтами, чтобы получить состояние State
3. Псевдослучайное число X возвращается как $(State[0] + State[1] * P + State[2] * P * P + \dots + State[7] * (P^7))$
4. State меняется как $State[1] = State[0], State[2] = State[1], \dots, State[7] = State[6], State[0] = X \% 256$

Выводы

Статистические тесты:

AES PRNG Sample Statistics:

Mean: 9053930279102768128.000000
Standard Deviation: 5419451002270816256.000000
Coefficient of Variation: 0.598574
Chi-Square Uniformity Test: 2297.066000
Chi-Square Randomness Test: 0

p-value for NIST tests:

Frequency Test: 0.000000
Block Frequency Test: 0.000000
Runs Test: 0.000000
Longest Run Test: 0.000000

Fibonacci PRNG Sample Statistics:

Mean: 9249605030853058560.000000
Standard Deviation: 5355264365073776640.000000
Coefficient of Variation: 0.578972
Chi-Square Uniformity Test: 19.576000
Chi-Square Randomness Test: 1

p-value for NIST tests:

Frequency Test: 0.545461
Block Frequency Test: 0.504522
Runs Test: 0.972948
Longest Run Test: 0.016950

Rolling PRNG Sample Statistics:

Mean: 9441715994641999872.000000
Standard Deviation: 5411431557224493056.000000
Coefficient of Variation: 0.573141
Chi-Square Uniformity Test: 1075.498000
Chi-Square Randomness Test: 0

p-value for NIST tests:

Frequency Test: 0.000000
Block Frequency Test: 0.999971
Runs Test: 0.000000
Longest Run Test: 0.000000

Лабораторная работа №3

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 AES_PRNG Struct Reference	5
3.1.1 Constructor & Destructor Documentation	5
3.1.1.1 AES_PRNG()	5
3.1.2 Member Function Documentation	6
3.1.2.1 generate()	6
3.2 FibonacciPRNG Struct Reference	6
3.2.1 Member Function Documentation	6
3.2.1.1 next()	6
3.3 RollingPRNG Struct Reference	7
3.3.1 Member Function Documentation	7
3.3.1.1 next()	7
4 File Documentation	9
4.1 src/aes_prng.h File Reference	9
4.1.1 Detailed Description	10
4.1.2 Function Documentation	10
4.1.2.1 AddKey()	10
4.1.2.2 generate_aes_sample()	10
4.1.2.3 ShiftRows()	11
4.1.2.4 SubBytes()	11
4.2 aes_prng.h	11
4.3 src/fib_prng.h File Reference	12
4.3.1 Detailed Description	12
4.3.2 Function Documentation	13
4.3.2.1 generate_fib_sample()	13
4.4 fib_prng.h	13
4.5 prng_tests.h	13
4.6 src/rolling_prng.h File Reference	14
4.6.1 Detailed Description	14
4.6.2 Function Documentation	15
4.6.2.1 generate_rolling_sample()	15
4.7 rolling_prng.h	15
4.8 src/statistic_tests.h File Reference	15
4.8.1 Detailed Description	16
4.8.2 Function Documentation	16

4.8.2.1 chi_square_uniformity_test()	16
4.8.2.2 cv()	17
4.8.2.3 isUniform()	17
4.8.2.4 mean()	17
4.8.2.5 stddev()	18
4.9 statistic_tests.h	18
4.10 src/Utils.h File Reference	18
4.10.1 Detailed Description	19
4.10.2 Function Documentation	19
4.10.2.1 measure_time_ms()	19
4.11 Utils.h	19
Index	21

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AES_PRNG		5
FibonacciPRNG		6
RollingPRNG		7

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/aes_prng.h	Объявление методов генерации псевдослучайных чисел с использованием алгоритма AES	9
src/fib_prng.h	Объявление методов генерации псевдослучайных чисел с использованием чисел Фибоначчи	12
src/prng_tests.h		13
src/rolling_prng.h	Объявление методов генерации псевдослучайных чисел с использованием полиномиального хеширования	14
src/statistic_tests.h	Объявление методов для проведения статистических тестов над случайными выборками: среднее, отклонение, коэффициент вариации, тест на равномерность и случайность с помощью критерия Хи-квадрат	15
src/utils.h	Вспомогательные функции	18

Chapter 3

Class Documentation

3.1 AES_PRNG Struct Reference

Public Member Functions

- [AES_PRNG](#) (const uint8_t seed[16])
Инициализировать генератор с помощью начального значения (seed).
- void [generate](#) (uint8_t output[16])
Сгенерировать псевдослучайное число, выполняя 10 раундов алгоритма AES.

Public Attributes

- uint8_t state [4][4]
Матрица состояния 4x4.

3.1.1 Constructor & Destructor Documentation

3.1.1.1 AES_PRNG()

```
AES_PRNG::AES_PRNG (  
    const uint8_t seed[16])
```

Инициализировать генератор с помощью начального значения (seed).

Parameters

seed	Начальное значение (16 байт)
------	------------------------------

3.1.2 Member Function Documentation

3.1.2.1 generate()

```
void AES_PRNG::generate (
    uint8_t output[16])
```

Сгенерировать псевдослучайное число, выполняя 10 раундов алгоритма AES.

Returns

Сгенерированное псевдослучайное число (16 байт)

The documentation for this struct was generated from the following files:

- [src/aes_prng.h](#)
- [src/aes_prng.cpp](#)

3.2 FibonacciPRNG Struct Reference

Public Member Functions

- [FibonacciPRNG \(uint64_t seed\)](#)
Конструктор для инициализации генератора с помощью начального значения (seed).
- [uint64_t next \(\)](#)
Генерация псевдослучайного числа

Public Attributes

- [uint64_t A](#)
- [uint64_t B](#)
Изначальный seed, который будет изменяться при генерации псевдослучайных чисел

3.2.1 Member Function Documentation

3.2.1.1 next()

```
uint64_t FibonacciPRNG::next ()
```

Генерация псевдослучайного числа

Returns

Сгенерированное псевдослучайное число

The documentation for this struct was generated from the following files:

- [src/fib_prng.h](#)
- [src/fib_prng.cpp](#)

3.3 RollingPRNG Struct Reference

Public Member Functions

- RollingPRNG (const uint8_t seed[8])
Конструктор для инициализации генератора с помощью начального значения (seed).
- uint64_t [next](#) ()
Генерация псевдослучайного числа

Public Attributes

- uint8_t state [8] = {0xAB, 0xCD, 0xEF, 0x12, 0x34, 0x56, 0x78, 0x90}
Состояние генератора, которое будет изменяться при генерации псевдослучайных чисел

Static Public Attributes

- static const uint64_t P = 0x100000001B3ULL
Произвольное простое число (FNV prime), используемое для полиномиального хэширования

3.3.1 Member Function Documentation

3.3.1.1 next()

```
uint64_t RollingPRNG::next ()
```

Генерация псевдослучайного числа

Returns

Сгенерированное псевдослучайное число

The documentation for this struct was generated from the following files:

- [src/rolling_prng.h](#)
- [src/rolling_prng.cpp](#)

Chapter 4

File Documentation

4.1 src/aes_prng.h File Reference

Объявление методов генерации псевдослучайных чисел с использованием алгоритма AES.

```
#include <stdint>
#include <vector>
```

Classes

- struct [AES_PRNG](#)

Functions

- void [SubBytes](#) (uint8_t state[4][4])
Заменить каждый байт из State на соответствующий байт согласно S.
- void [ShiftRows](#) (uint8_t state[4][4])
Циклически сдвинуть влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3.
- void [AddKey](#) (uint8_t state[4][4])
Поэлементно прибавить Key к State используя XOR.
- std::vector< uint64_t > [generate_aes_sample](#) (const uint8_t seed[16], size_t size)
Найти выборку по seed.

4.1.1 Detailed Description

Объявление методов генерации псевдослучайных чисел с использованием алгоритма AES.

1. Задан параметр Key - матрица байт 4x4 с заранее случайно сгенерированными числами, S - таблица замены, где для каждого байта определено значение, на которое он заменяется
2. Изначальное состояние (seed, 16 байт) записывается в матрицу 4x4. Назовем матрицу состояния State
3. 10 раз повторяется следующее:
 - (a) Операция SubBytes заменяет каждый байт из State на соответствующий байт согласно S
 - (b) ShiftRows циклически сдвигает влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3
 - (c) AddKey поэлементно прибавляет Key к State используя XOR
4. Состояние возвращается как псевдослучайное число

4.1.2 Function Documentation

4.1.2.1 AddKey()

```
void AddKey (
    uint8_t state[4][4])
```

Поэлементно прибавить Key к State используя XOR.

Parameters

state	Входная матрица состояния 4x4
-------	-------------------------------

4.1.2.2 generate_aes_sample()

```
std::vector< uint64_t > generate_aes_sample (
    const uint8_t seed[16],
    size_t size)
```

Найти выборку по seed.

Parameters

seed	Начальное значение (16 байт)
size	Размер выборки

Returns

Вектор с выборкой, сгенерированной на основе seed

4.1.2.3 ShiftRows()

```
void ShiftRows (
    uint8_t state[4][4])
```

Циклически сдвинуть влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3.

Parameters

state	Входная матрица состояния 4x4
-------	-------------------------------

4.1.2.4 SubBytes()

```
void SubBytes (
    uint8_t state[4][4])
```

Заменить каждый байт из State на соответствующий байт согласно S.

Parameters

state	Входная матрица состояния 4x4
-------	-------------------------------

4.2 aes_prng.h

[Go to the documentation of this file.](#)

```
00001
00016 #pragma once
00017
00018 #include <stdint>
00019 #include <vector>
00020
00021 static const uint8_t key[4][4] = {
00022     {0x72, 0xdd, 0xcb, 0x1f},
00023     {0x7a, 0xb4, 0x7b, 0xa0},
00024     {0x6f, 0x32, 0x48, 0xef},
00025     {0xa6, 0xad, 0x8e, 0xe4}};
00026
00027 static const uint8_t sbox[256] = {
00028     // 0 1 2 3 4 5 6 7 8 9 A B C D E F
00029     0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
00030     0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
00031     0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
00032     0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
00033     0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
00034     0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
00035     0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
00036     0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
00037     0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
00038     0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
00039     0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
00040     0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
00041     0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
00042     0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
00043     0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
00044     0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16};
00045
00050 void SubBytes(uint8_t state[4][4]);
00051
00056 void ShiftRows(uint8_t state[4][4]);
00057
00062 void AddKey(uint8_t state[4][4]);
00063
00064 struct AES_PRNG
```

```
00065 {
00066     uint8_t state[4][4];
00067
00072     AES_PRNG(const uint8_t seed[16]);
00073
00078     void generate(uint8_t output[16]);
00079 };
00080
00087 std::vector<uint64_t> generate_aes_sample(const uint8_t seed[16], size_t size);
```

4.3 src/fib_prng.h File Reference

Объявление методов генерации псевдослучайных чисел с использованием чисел Фибоначчи

```
#include <cstdint>
#include <vector>
```

Classes

- struct [FibonacciPRNG](#)

Functions

- `std::vector< uint64_t > generate\_fib\_sample (uint64_t seed, size_t num_samples)`
Найти выборку по seed.

Variables

- `const uint64_t A_initial = 0x123456789abcdef0`
Изначальный параметр генератора

4.3.1 Detailed Description

Объявление методов генерации псевдослучайных чисел с использованием чисел Фибоначчи

1. Задан параметр A - изначальный параметр генератора
2. Изначальный seed - число, кладется в переменную B
3. Состояние вычисляется согласно модифицированной формулы чисел Фибоначчи: $A, B = B, A * A \wedge B * B$
4. B возвращается как псевдослучайное число

4.3.2 Function Documentation

4.3.2.1 generate_fib_sample()

```
std::vector< uint64_t > generate_fib_sample (
    uint64_t seed,
    size_t num_samples)
```

Найти выборку по seed.

Parameters

seed	Начальное значение (16 байт)
num_samples	Размер выборки

Returns

Вектор с выборкой, сгенерированной на основе seed

4.4 fib_prng.h

[Go to the documentation of this file.](#)

```
00001
00010 #pragma once
00011
00012 #include <cstdint>
00013 #include <vector>
00014
00015 const uint64_t A_initial = 0x123456789abcdef0;
00016
00017 struct FibonacciPRNG
00018 {
00019     uint64_t A, B;
00020
00021     FibonacciPRNG(uint64_t seed) : A(A_initial), B(seed) {}
00022
00023     uint64_t next();
00024 };
00025
00026 std::vector<uint64_t> generate_fib_sample(uint64_t seed, size_t num_samples);
```

4.5 prng_tests.h

```
00001
00006 #pragma once
00007
00008 #include <cstdint>
00009 #include <vector>
00010
00016 long double frequencyTest(const std::vector<uint64_t> &data);
00017
00024 long double blockFrequencyTest(const std::vector<uint64_t> &data, int block_size);
00025
00031 long double runsTest(const std::vector<uint64_t> &data);
00032
00038 long double longestRunTest(const std::vector<uint64_t> &data);
00039
00046 long double overlappingTemplateTest(const std::vector<uint64_t> &data, int template_size);
```

4.6 src/rolling_prng.h File Reference

Объявление методов генерации псевдослучайных чисел с использованием полиномиального хэширования

```
#include <cstdint>
#include <vector>
```

Classes

- struct [RollingPRNG](#)

Functions

- `std::vector< uint64_t > generate\_rolling\_sample (const uint8_t seed[8], size_t num_samples)`
Найти выборку по seed.

4.6.1 Detailed Description

Объявление методов генерации псевдослучайных чисел с использованием полиномиального хэширования

1. Задается seed - 8 байт
2. seed побайтово складывается с заранее заданными случайными 8 байтами, чтобы получить состояние State
3. Псевдослучайное число X возвращается как $(State[0] + State[1] * P + State[2] * P * P + \dots + State[7] * (P^7))$
4. State меняется как $State[1] = State[0]$, $State[2] = State[1]$, ..., $State[7] = State[6]$, $State[0] = X \% 256$

4.6.2 Function Documentation

4.6.2.1 generate_rolling_sample()

```
std::vector< uint64_t > generate_rolling_sample (
    const uint8_t seed[8],
    size_t num_samples)
```

Найти выборку по seed.

Parameters

seed	Начальное значение (8 байт)
num_samples	Размер выборки

Returns

Вектор с выборкой, сгенерированной на основе seed

4.7 rolling_prng.h

[Go to the documentation of this file.](#)

```
00001
00013 #pragma once
00014
00015 #include <stdint>
00016 #include <vector>
00017
00018 struct RollingPRNG
00019 {
00020     const static uint64_t P = 0x100000001B3ULL;
00021     uint8_t state[8] = {0xAB, 0xCD, 0xEF, 0x12, 0x34, 0x56, 0x78, 0x90};
00022
00023     RollingPRNG(const uint8_t seed[8]);
00024
00029     uint64_t next();
00030 };
00031
00032
00039 std::vector<uint64_t> generate_rolling_sample(const uint8_t seed[8], size_t num_samples);
```

4.8 src/statistic_tests.h File Reference

Объявление методов для проведения статистических тестов над случайными выборками:↔
среднее, отклонение, коэффициент вариации, тест на равномерность и случайность с помощью критерия Хи-квадрат

```
#include <stdint>
#include <vector>
```

Functions

- long double [mean](#) (const std::vector< uint64_t > &data)
Вычисление среднего значения выборки
- long double [stddev](#) (const std::vector< uint64_t > &data)
Вычисление стандартного отклонения выборки
- long double [cv](#) (const std::vector< uint64_t > &data)
Вычисление коэффициента вариации выборки
- long double [chi_square_uniformity_test](#) (const std::vector< uint64_t > &data, int num_bins)
Проведение теста на равномерность выборки с помощью критерия Хи-квадрат
- bool [isUniform](#) (long double chiSquare, int num_bins)
Проверить, соответствует ли выборка распределению, близкому к равномерному

4.8.1 Detailed Description

Объявление методов для проведения статистических тестов над случайными выборками:↔ среднее, отклонение, коэффициент вариации, тест на равномерность и случайность с помощью критерия Хи-квадрат

4.8.2 Function Documentation

4.8.2.1 chi_square_uniformity_test()

```
long double chi_square_uniformity_test (  
    const std::vector< uint64_t > & data,  
    int num_bins)
```

Проведение теста на равномерность выборки с помощью критерия Хи-квадрат

Parameters

data	Вектор с данными выборки
num_bins	Количество интервалов (корзин) для разбиения данных

Returns

Значение статистики Хи-квадрат для теста на равномерность

4.8.2.2 cv()

```
long double cv (  
    const std::vector< uint64_t > & data)
```

Вычисление коэффициента вариации выборки

Parameters

data	Вектор с данными выборки
------	--------------------------

Returns

Коэффициент вариации выборки

4.8.2.3 isUniform()

```
bool isUniform (  
    long double chiSquare,  
    int num_bins)
```

Проверить, соответствует ли выборка распределению, близкому к равномерному

Parameters

chiSquare	Значение статистики Хи-квадрат
num_bins	Количество интервалов (корзин) для разбиения данных

Returns

true, если выборка соответствует равномерному распределению, иначе false

4.8.2.4 mean()

```
long double mean (  
    const std::vector< uint64_t > & data)
```

Вычисление среднего значения выборки

Parameters

data	Вектор с данными выборки
------	--------------------------

Returns

Среднее значение выборки

4.8.2.5 stddev()

```
long double stddev (
    const std::vector< uint64_t > & data)
```

Вычисление стандартного отклонения выборки

Parameters

data	Вектор с данными выборки
------	--------------------------

Returns

Стандартное отклонение выборки

4.9 statistic_tests.h

[Go to the documentation of this file.](#)

```
00001
00007 #pragma once
00008
00009 #include <cstdint>
00010 #include <vector>
00011
00017 long double mean(const std::vector<uint64_t> &data);
00018
00024 long double stddev(const std::vector<uint64_t> &data);
00025
00031 long double cv(const std::vector<uint64_t> &data);
00032
00039 long double chi_square_uniformity_test(const std::vector<uint64_t> &data, int num_bins);
00040
00047 bool isUniform(long double chiSquare, int num_bins);
```

4.10 src/utils.h File Reference

Вспомогательные функции

```
#include <chrono>
#include <cstdint>
#include <cstdint>
#include <functional>
#include <vector>
```

Functions

- double [measure_time_ms](#) (std::function< std::vector< uint64_t >(size_t)> gen_func, size_t size)
Измерение времени генерации выборки псевдослучайных чисел

4.10.1 Detailed Description

Вспомогательные функции

4.10.2 Function Documentation

4.10.2.1 measure_time_ms()

```
double measure_time_ms (  
    std::function< std::vector< uint64_t >(size_t)> gen_func,  
    size_t size)
```

Измерение времени генерации выборки псевдослучайных чисел

Parameters

gen_func	Функция-генератор, принимающая размер выборки и возвращающая её
size	Размер выборки

Returns

Время генерации (в миллисекундах)

4.11 utils.h

[Go to the documentation of this file.](#)

```
00001  
00005  
00006 #pragma once  
00007  
00008 #include <chrono>  
00009 #include <cstdint>  
00010 #include <functional>  
00011 #include <vector>  
00012  
00020 double measure_time_ms(std::function<std::vector<uint64_t>(size_t)> gen_func, size_t size);
```


Index

- AddKey
 - aes_prng.h, [10](#)
- AES_PRNG, [5](#)
 - AES_PRNG, [5](#)
 - generate, [6](#)
- aes_prng.h
 - AddKey, [10](#)
 - generate_aes_sample, [10](#)
 - ShiftRows, [10](#)
 - SubBytes, [11](#)
- chi_square_uniformity_test
 - statistic_tests.h, [16](#)
- cv
 - statistic_tests.h, [16](#)
- fib_prng.h
 - generate_fib_sample, [13](#)
- FibonacciPRNG, [6](#)
 - next, [6](#)
- generate
 - AES_PRNG, [6](#)
- generate_aes_sample
 - aes_prng.h, [10](#)
- generate_fib_sample
 - fib_prng.h, [13](#)
- generate_rolling_sample
 - rolling_prng.h, [15](#)
- isUniform
 - statistic_tests.h, [17](#)
- mean
 - statistic_tests.h, [17](#)
- measure_time_ms
 - utils.h, [19](#)
- next
 - FibonacciPRNG, [6](#)
 - RollingPRNG, [7](#)
- rolling_prng.h
 - generate_rolling_sample, [15](#)
- RollingPRNG, [7](#)
 - next, [7](#)
- ShiftRows
 - aes_prng.h, [10](#)
- src/aes_prng.h, [9](#), [11](#)
- src/fib_prng.h, [12](#), [13](#)
- src/prng_tests.h, [13](#)
- src/rolling_prng.h, [14](#), [15](#)
- src/statistic_tests.h, [15](#), [18](#)
- src/utils.h, [18](#), [19](#)
- statistic_tests.h
 - chi_square_uniformity_test, [16](#)
 - cv, [16](#)
 - isUniform, [17](#)
 - mean, [17](#)
 - stddev, [17](#)
- stddev
 - statistic_tests.h, [17](#)
- SubBytes
 - aes_prng.h, [11](#)
- utils.h
 - measure_time_ms, [19](#)